
LINUX

man Command

man command in Linux is used to display the user manual of any command that we can run on the terminal. It provides a detailed view of the command which includes

- NAME,
- SYNOPSIS,
- DESCRIPTION,
- OPTIONS,
- EXIT STATUS,
- RETURN VALUES,
- ERRORS,
- FILES,
- VERSIONS,
- EXAMPLES,
- AUTHORS and SEE ALSO.

man Command

Every manual is divided into the following sections:

- Executable programs or shell commands
- System calls (functions provided by the kernel)
- Library calls (functions within program libraries
- Games
- Special files (usually found in /dev)
- File formats and conventions eg /etc/passwd
- Miscellaneous (including macro packages and conventions), e.g. groff(7)
- System administration commands (usually only for root)
- Kernel routines [Non standard]

man Command

Syntax :

```
$man [OPTION]... [COMMAND NAME]...
```

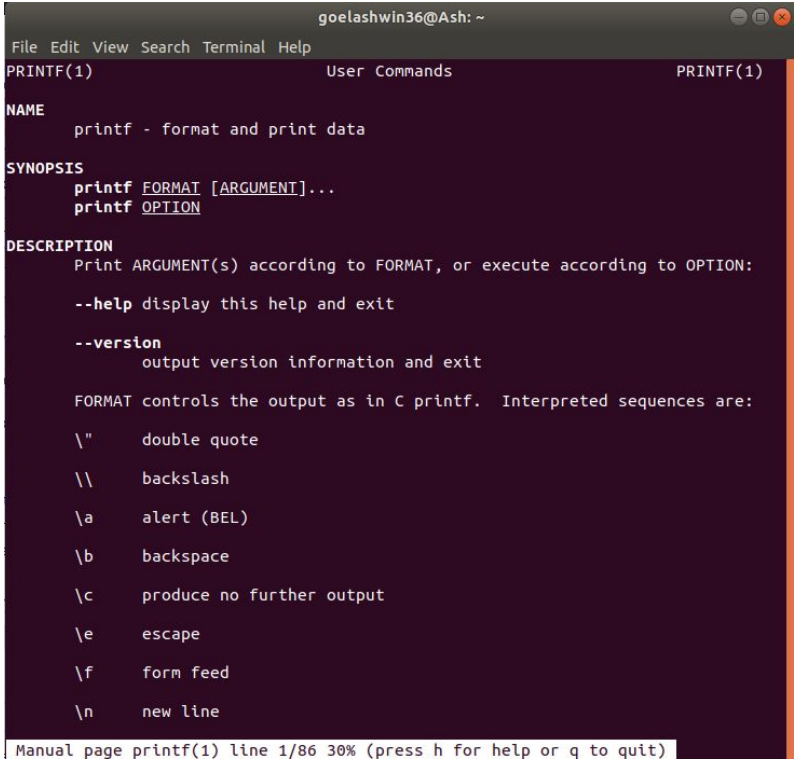
Options and Examples

1. No Option: It displays the whole manual of the command.

```
$ man [COMMAND NAME]
```

```
$ man printf
```

In this example, manual pages of the command '*printf*' are simply returned.



man Command

Syntax :

```
$man [OPTION]... [COMMAND NAME]...
```

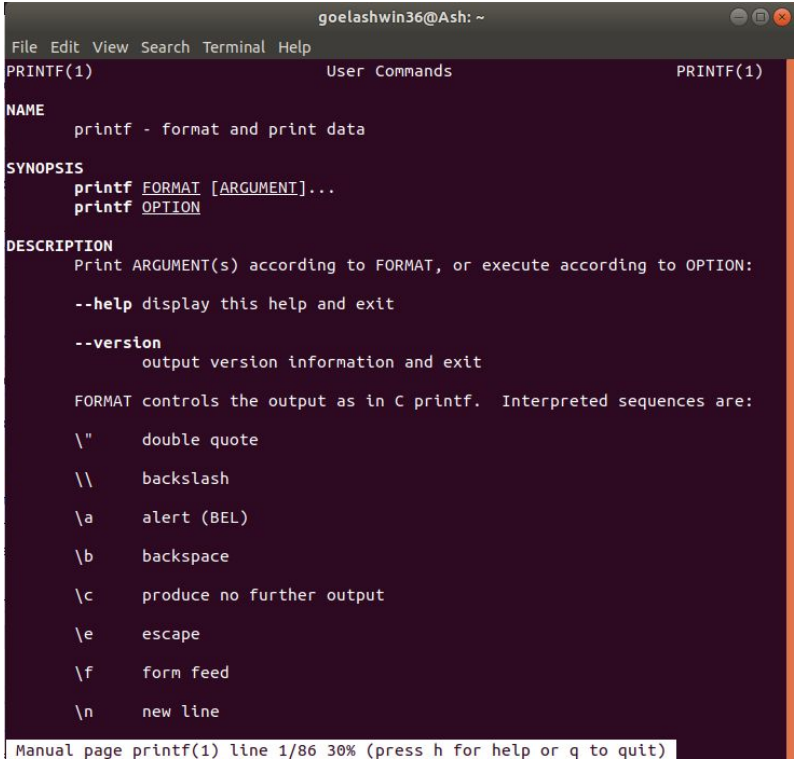
Options and Examples

1. No Option: It displays the whole manual of the command.

```
$ man [COMMAND NAME]
```

```
$ man printf
```

In this example, manual pages of the command '*printf*' are simply returned.



man Command

2. Section-num: Since a manual is divided into multiple sections so this option is used to display only a specific section of a manual.

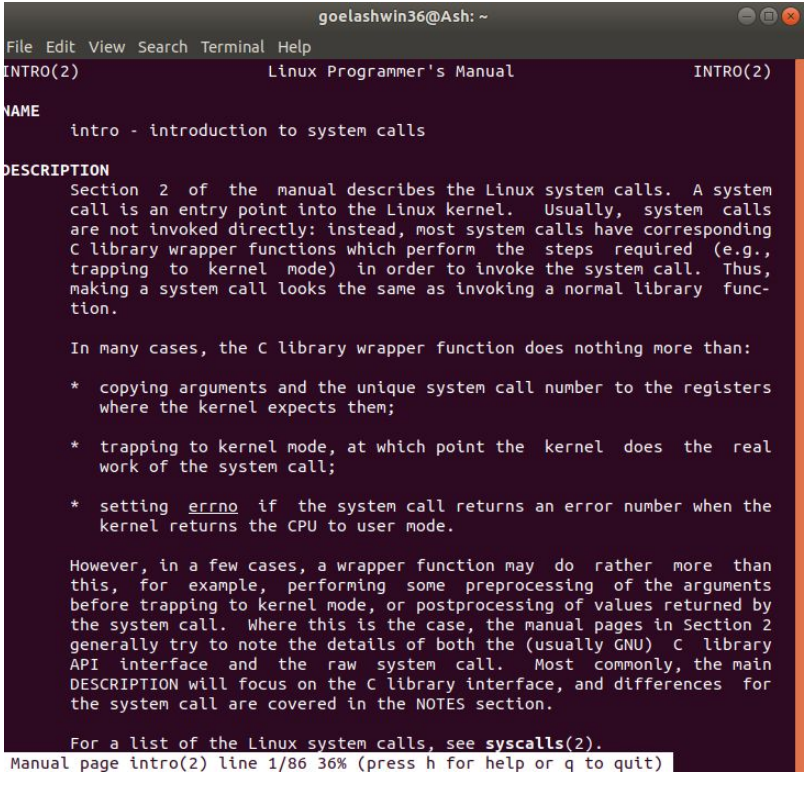
Syntax :

$ man [SECTION-NUM] [COMMAND NAME]

Example:

$ man 2 intro

In this example, the manual pages of command ‘intro‘ are returned which lies in the section 2



man Command

3. -f option: One may not be able to remember the sections in which a command is present. So this option gives the section in which the given command is present.

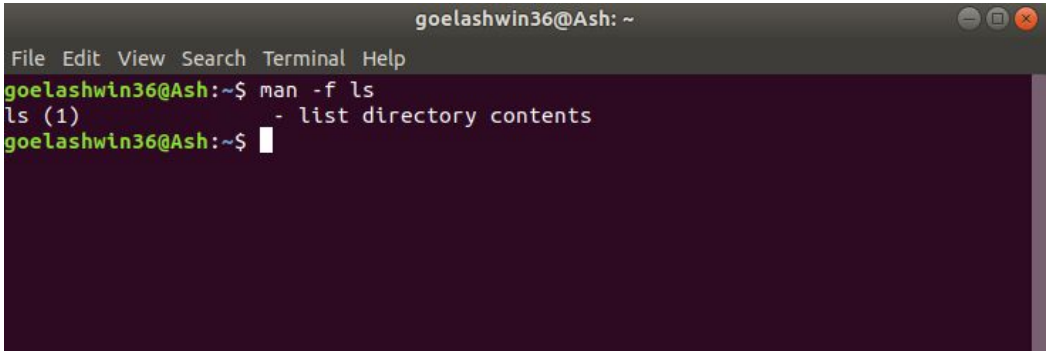
Syntax:

$ man -f [COMMAND NAME]

Example:

$ man -f ls

In this example, the command ‘ls‘ is returned with its section number.



man Command

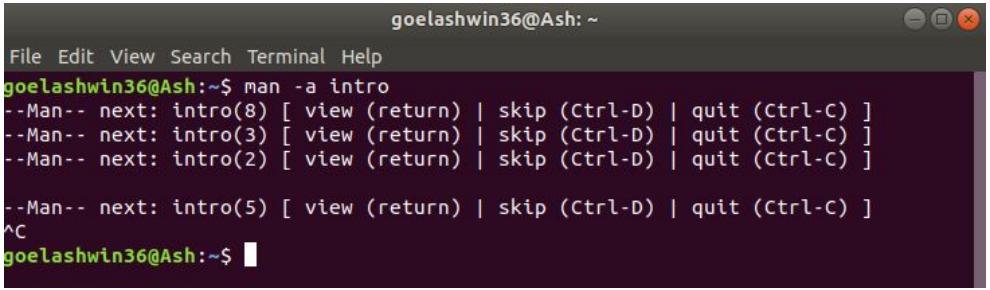
4. -a option: This option helps us to display all the available intro manual pages in succession.

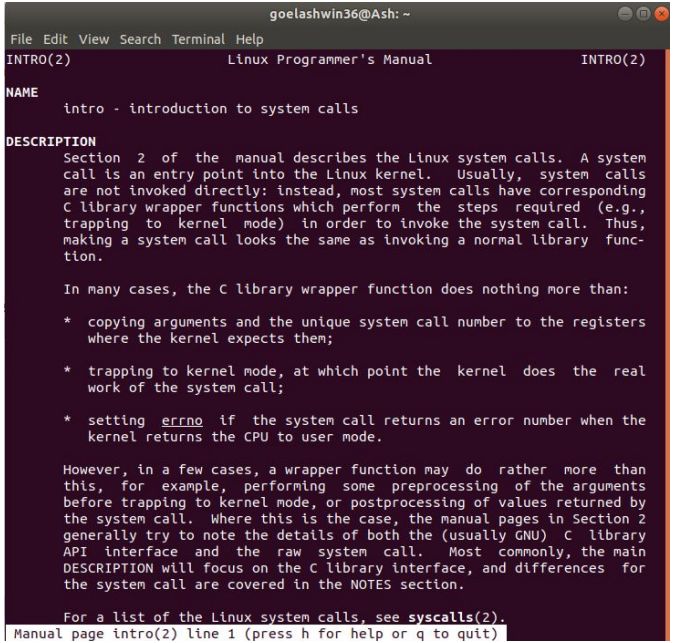
Syntax:

$ man -a [COMMAND NAME]

Example:

$ man -a intro





In this example you can move through the manual pages(sections) i.e either reading(by pressing Enter) or skipping(by pressing ctrl+D) or exiting(by pressing ctrl+C).

man Command

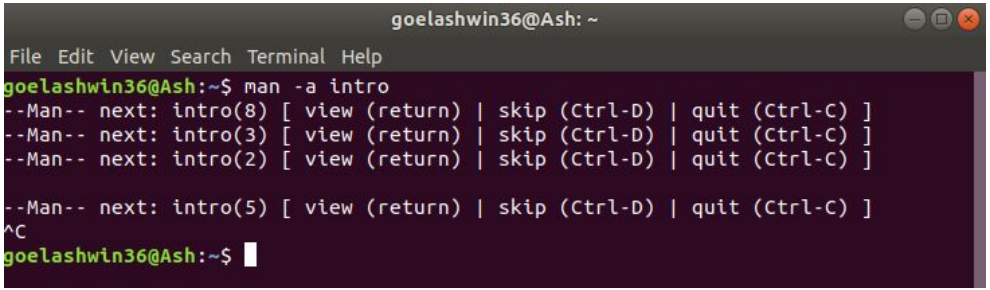
4. -a option: This option helps us to display all the available intro manual pages in succession.

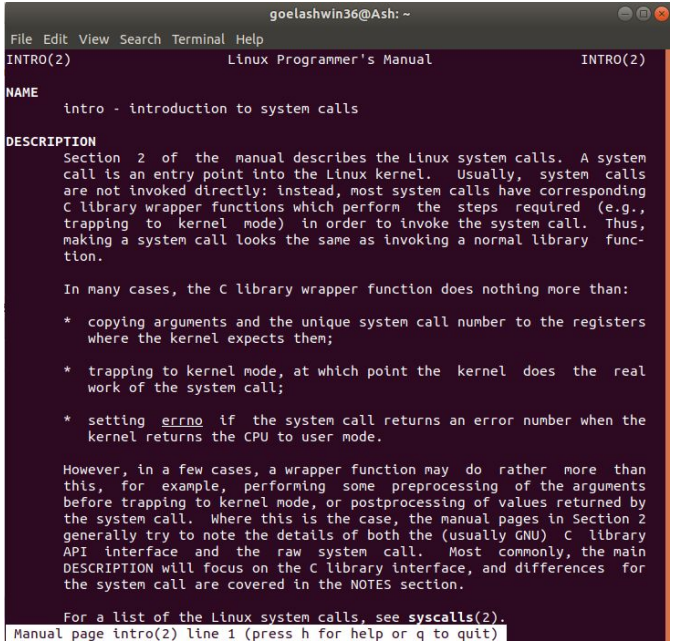
Syntax:

$ man -a [COMMAND NAME]

Example:

$ man -a intro





In this example you can move through the manual pages(sections) i.e either reading(by pressing Enter) or skipping(by pressing ctrl+D) or exiting(by pressing ctrl+C).

man Command

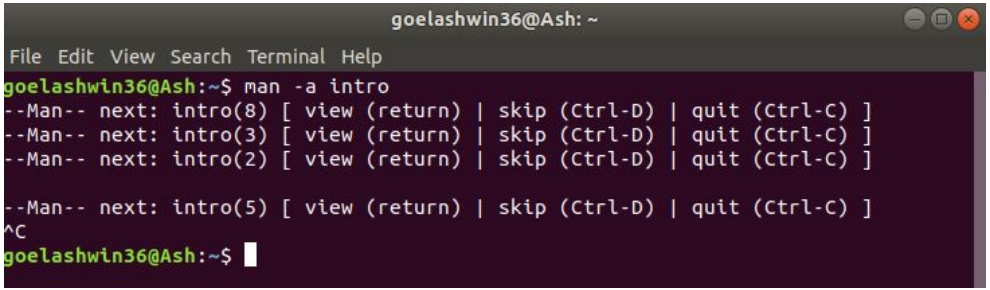
4. -a option: This option helps us to display all the available intro manual pages in succession.

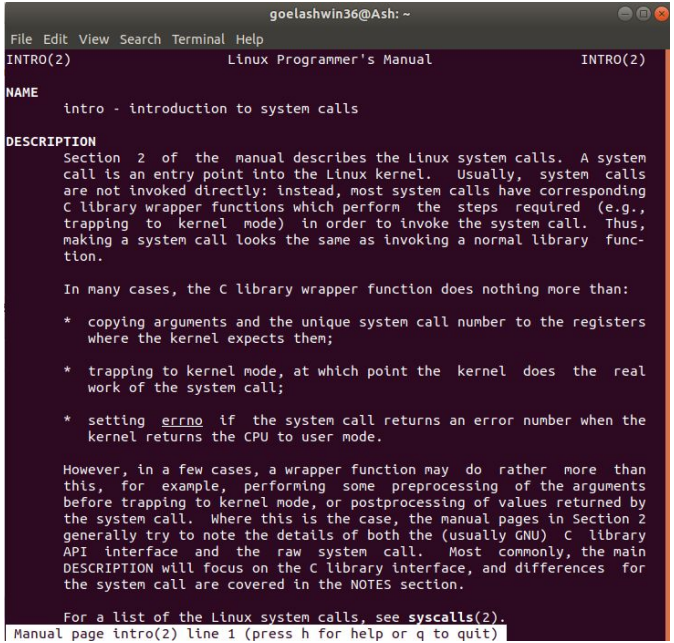
Syntax:

$ man -a [COMMAND NAME]

Example:

$ man -a intro





In this example you can move through the manual pages(sections) i.e either reading(by pressing Enter) or skipping(by pressing ctrl+D) or exiting(by pressing ctrl+C).

man Command

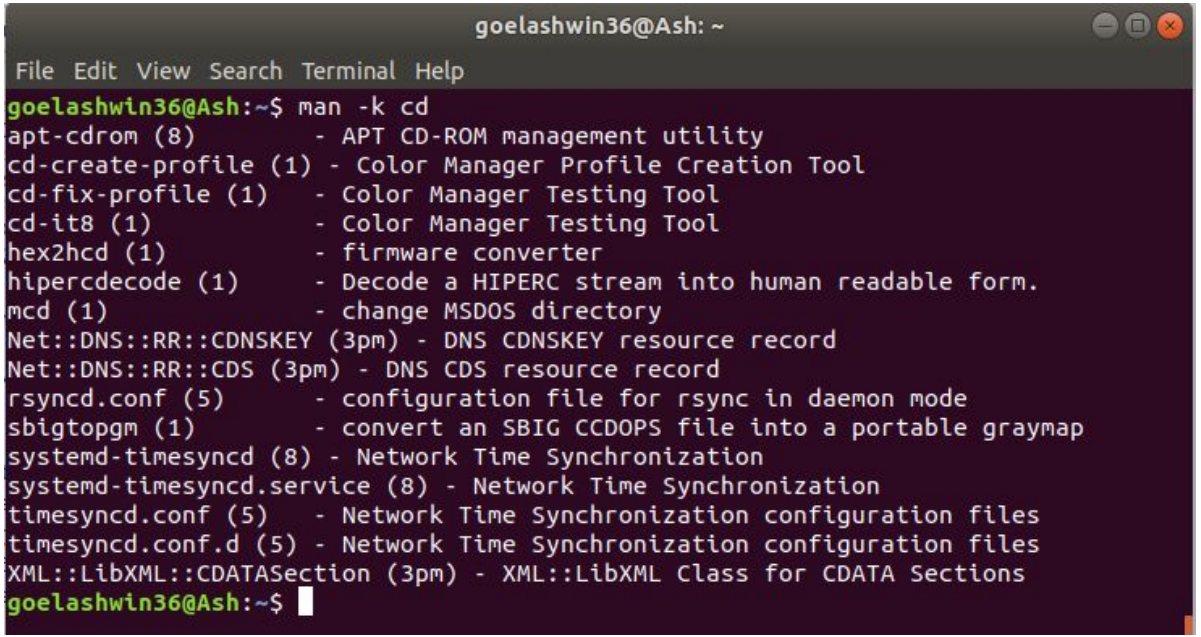
5. -k option: This option searches the given command as a regular expression in all the manuals and it returns the manual pages with the section number in which it is found.

Syntax:

$ man -k [COMMAND NAME]

Example:

$ man -k cd



The command ‘cd‘ is searched in all the manual pages by considering it as a regular expression.

man Command

6. -w option: This option returns the location in which the manual page of a given command is present.

Syntax:

$ man -w [COMMAND NAME]

Example:

$ man -w ls

The location of command ‘ls‘ is returned.

man Command

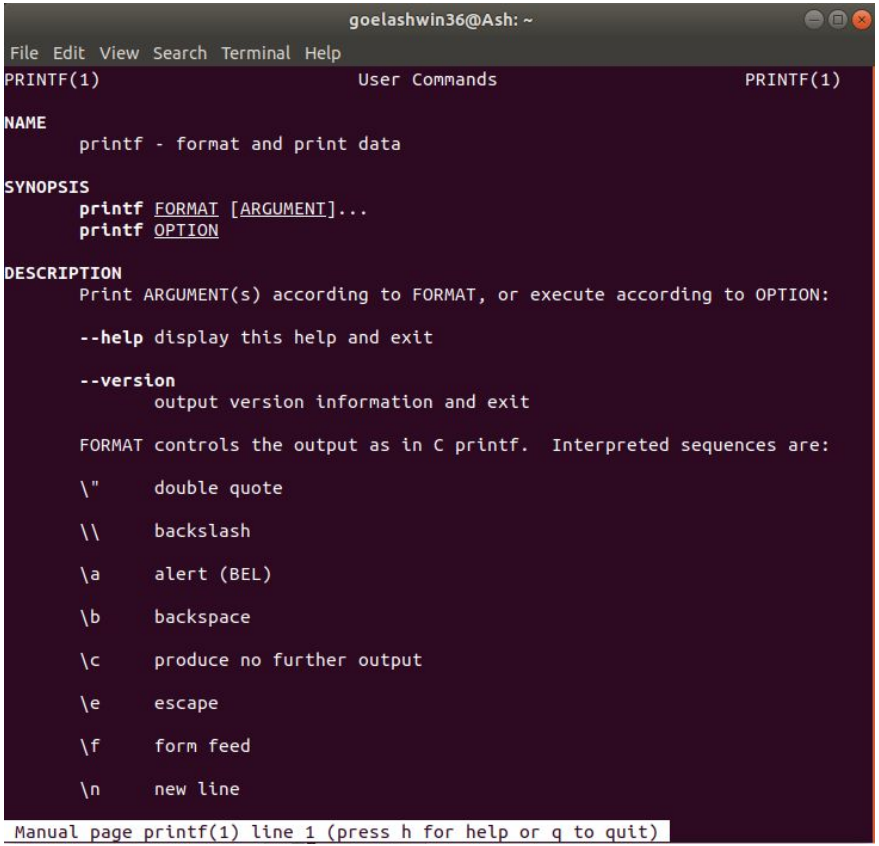
7. -I option: It considers the command as case sensitive.

Syntax:

$ man -I [COMMAND NAME]

Example:

$ man -I printf



The command '*printf*' is taken as case-sensitive i.e '*printf*' returns the manual pages but '*Printf*' gives error.

whatis Command

Describes what function a command performs.

Syntax:

whatis [**-M** *PathName*] *Command …*

Description

The **whatis** command looks up a given command, system call, library function, or special file name, as specified by the *Command* parameter, from a database you create using the **catman -w** command. The **whatis** command displays the header line from the manual section. You can then issue the **man** command to obtain additional information.

The **whatis** command is equivalent to using the **man -f** command.

Flags :

Item	Description
-M *PathName*	Specifies an alternative search path. The search path is specified by the *PathName* parameter, and is a colon-separated list of directories in which the **whatis** command expects to find the standard manual subdirectories.

whatis Command

Example:

$ whatis ls

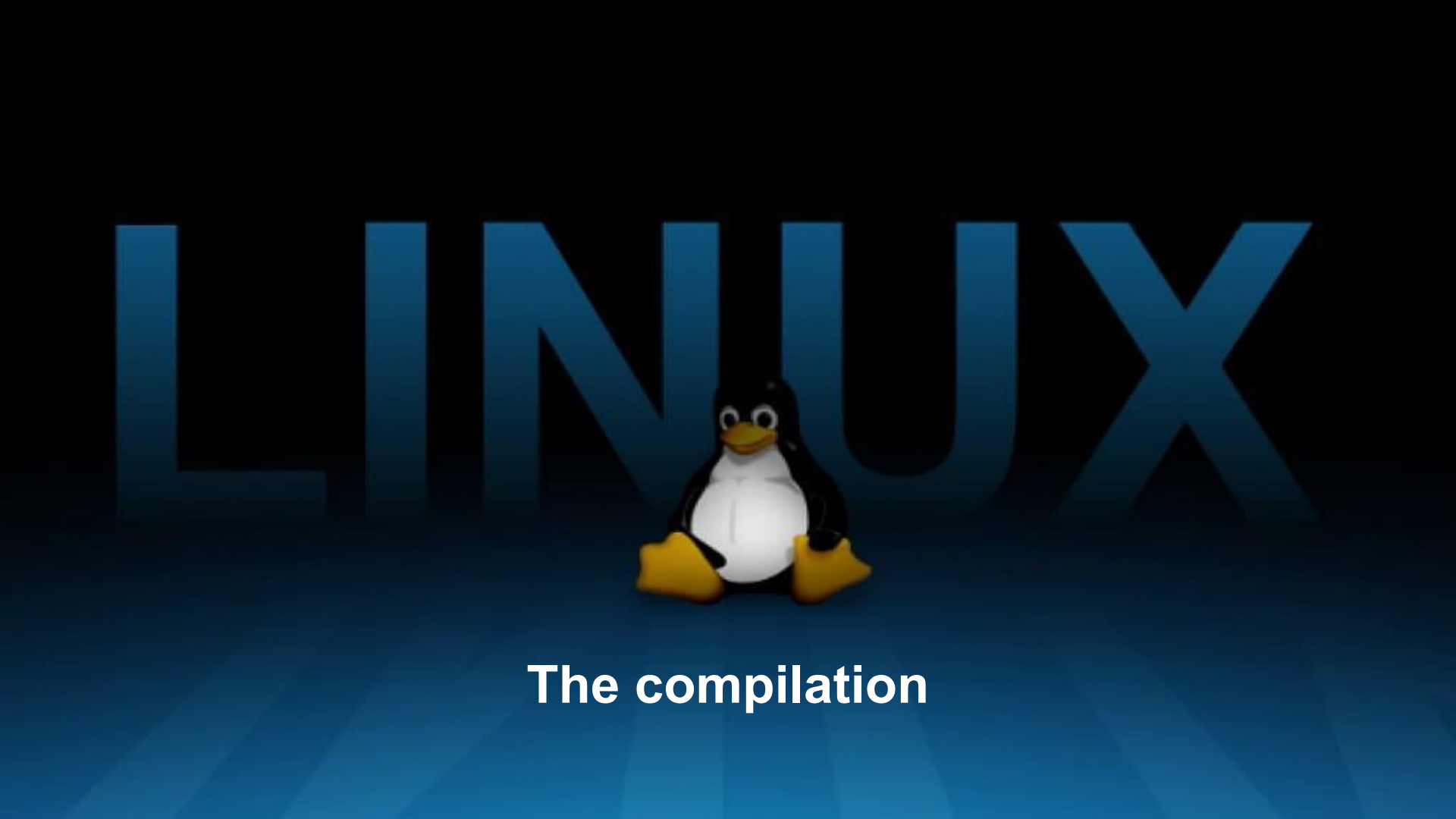
```
MacBook-Air-di-Valeria-10:~ valerialukaj$ whatis ls
git-ls-files(1)          - Show information about files in the index and the working tree
git-ls-remote(1)         - List references in a remote repository
git-ls-tree(1)           - List the contents of a tree object
git-mktree(1)            - Build a tree-object from ls-tree formatted text
builtin(1), !(1), %(1), .(1), :(1), @(1), {(1), }(1), alias(1), alloc(1), bg(1), bind(1), bindkey(1), break(1), breaksw(1), builtins(1), case(1), cd(1), chdir(1), command(1), com
plete(1), continue(1), default(1), dirs(1), do(1), done(1), echo(1), echotc(1), elif(1), else(1), end(1), endif(1), endsw(1), esac(1), eval(1), exec(1), exit(1), export(1), false
(1), fc(1), fg(1), filetest(1), fi(1), for(1), foreach(1), getopts(1), glob(1), goto(1), hash(1), hashstat(1), history(1), hup(1), if(1), jobid(1), jobs(1), kill(1), limit(1), lo
cal(1), log(1), login(1), logout(1), ls-F(1), nice(1), nohup(1), notify(1), onintr(1), popd(1), printenv(1), pushd(1), pwd(1), read(1), readonly(1), rehash(1), repeat(1), return(
1), sched(1), set(1), setenv(1), settc(1), setty(1), setvar(1), shift(1), source(1), stop(1), suspend(1), switch(1), telltc(1), test(1), then(1), time(1), times(1), trap(1), true
(1), type(1), ulimit(1), umask(1), unalias(1), uncomplete(1), unhash(1), unlimit(1), unset(1), unsetenv(1), until(1), wait(1), where(1), which(1), while(1) - shell built-in comma
nds
ls(1)                    - list directory contents
```

whatis Command

Example:

$ whatis ls

```
MacBook-Air-di-Valeria-10:~ valerialukaj$ whatis ls
git-ls-files(1)          - Show information about files in the index and the working tree
git-ls-remote(1)         - List references in a remote repository
git-ls-tree(1)           - List the contents of a tree object
git-mktree(1)            - Build a tree-object from ls-tree formatted text
builtin(1), !(1), %(1), .(1), :(1), @(1), {(1), }(1), alias(1), alloc(1), bg(1), bind(1), bindkey(1), break(1), breaksw(1), builtins(1), case(1), cd(1), chdir(1), command(1), com
plete(1), continue(1), default(1), dirs(1), do(1), done(1), echo(1), echotc(1), elif(1), else(1), end(1), endif(1), endsw(1), esac(1), eval(1), exec(1), exit(1), export(1), false
(1), fc(1), fg(1), filetest(1), fi(1), for(1), foreach(1), getopts(1), glob(1), goto(1), hash(1), hashstat(1), history(1), hup(1), if(1), jobid(1), jobs(1), kill(1), limit(1), lo
cal(1), log(1), login(1), logout(1), ls-F(1), nice(1), nohup(1), notify(1), onintr(1), popd(1), printenv(1), pushd(1), pwd(1), read(1), readonly(1), rehash(1), repeat(1), return(
1), sched(1), set(1), setenv(1), settc(1), setty(1), setvar(1), shift(1), source(1), stop(1), suspend(1), switch(1), telltc(1), test(1), then(1), time(1), times(1), trap(1), true
(1), type(1), ulimit(1), umask(1), unalias(1), uncomplete(1), unhash(1), unlimit(1), unset(1), unsetenv(1), until(1), wait(1), where(1), which(1), while(1) - shell built-in comma
nds
ls(1)                    - list directory contents
```



The compilation

Linux Compiler

- gcc(GNU cc) is the compiler available under Linux for programs written in C, C++
- The gcc compiler provides the programmer with extensive control over the compilation process.
- The possible phases of a build process are as follows:
  - Preprocessing stage
  - actual compilation
  - Assembly
  - Linking
- It handles several C dialects such as ANSI-C or traditional KR-C.
- Control the amount and type of debugging information
- Code optimization

Linux Compiler

Install GCC

To confirm whether GCC is already installed on your system, use the `gcc` command:

```
$ gcc --version
```

If necessary, install GCC using your packaging manager. On Fedora-based systems, use `dnf`:

```
$ sudo dnf install gcc libgcc
```

On Debian-based systems, use `apt`:

```
$ sudo apt install build-essential
```

Linux Compiler

Install GCC

To confirm whether GCC is already installed on your system, use the `gcc` command:

```
$ gcc --version
```

If necessary, install GCC using your packaging manager. On Fedora-based systems, use `dnf`:

```
$ sudo dnf install gcc libgcc
```

On Debian-based systems, use `apt`:

```
$ sudo apt install build-essential
```

Linux Compiler

Install GCC

After installation, if you want to check where GCC is installed, then use:

```
$ whereis gcc
```

Linux Compiler

Simple C program using GCC

Here's a simple C program to demonstrate how to compile code using GCC. Open your favorite text editor and paste in this code:

```
// hellogcc.c

#include <stdio.h>

int main() {

        printf("Hello, GCC!\n");

return 0;

}
```

Linux Compiler

Save the file as `hellogcc.c` and then compile it:

```
$ ls
```

```
MacBook-Air-di-Valeria-10:gcc valerialukaj$ ls
hellogcc.c
MacBook-Air-di-Valeria-10:gcc valerialukaj$
```

```
$ gcc hellogcc.c
```

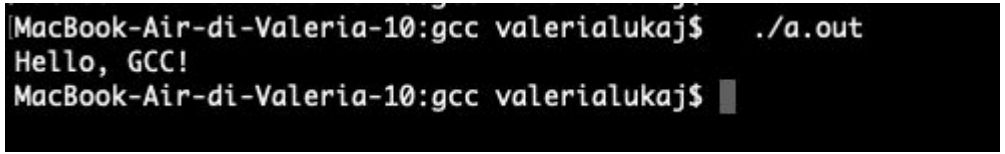
```
$ ls -1
```

```
MacBook-Air-di-Valeria-10:gcc valerialukaj$ ls -1
a.out
hellogcc.c
```

Linux Compiler

As you can see, `a.out` is the default executable generated as a result of compilation. To see the output of your newly-compiled application, just run it as you would any local binary:

```
$ ./a.out
```

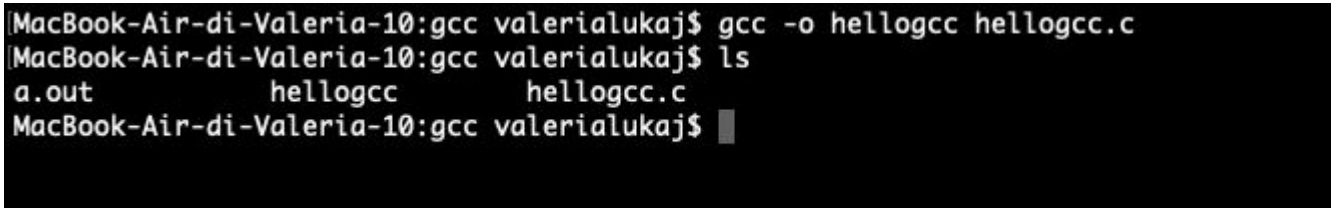


Linux Compiler

Name the output file

The filename `a.out` isn't very descriptive, so if you want to give a specific name to your executable file, you can use the `-o` option:

```
$ gcc -o hellogcc hellogcc.c
```



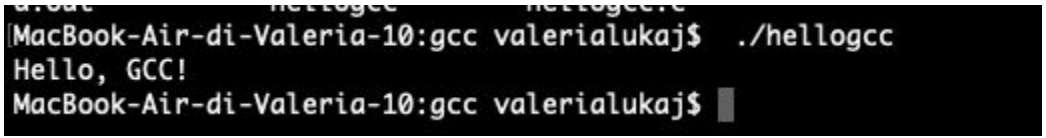
Linux Compiler

Name the output file

This option is useful when developing a large application that needs to compile multiple C source files.

```
$ ./hellogcc

Hello, GCC!
```



Intermediate steps in GCC compilation

There are actually four steps to compiling, even though GCC performs them automatically in simple use-cases.

- **Pre-Processing:** The GNU C Preprocessor (`cpp`) parses the headers (#include statements), expands macros (#define statements), and generates an intermediate file such as `hellogcc.i` with expanded source code.

- Compilation: During this stage, the compiler converts pre-processed source code into assembly code for a specific CPU architecture. The resulting assembly file is named with a `.s` extension, such as `hellogcc.s` in this example.

Intermediate steps in GCC compilation

There are actually four steps to compiling, even though GCC performs them automatically in simple use-cases.

- **Assembly:** The assembler (`as`) converts the assembly code into machine code in an object file, such as `hellogcc.o`.

- **Linking:** The linker (`ld`) links the object code with the library code to produce an executable file, such as `hellogcc`.

Linux Compiler

When running GCC, use the `-v` option to see each step in detail.

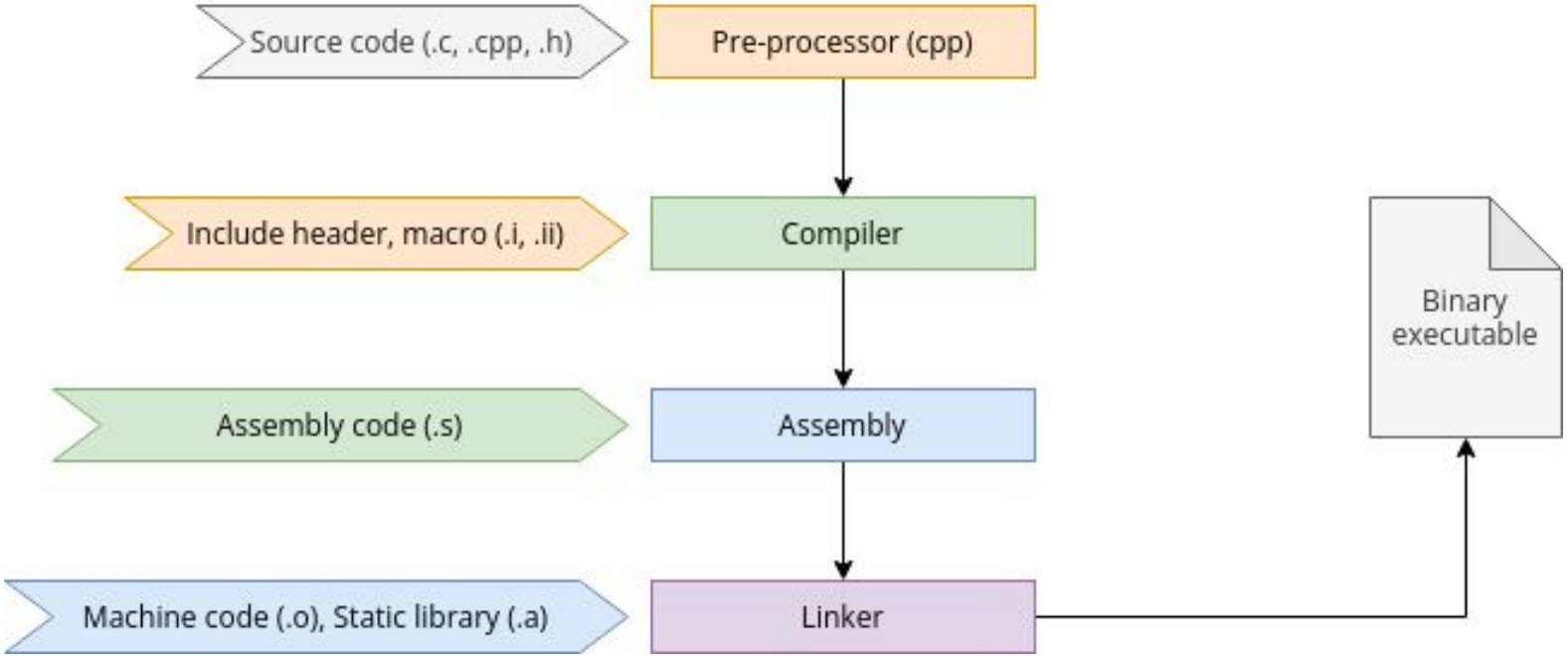
```
$ gcc -v -o hellogcc hellogcc.c
```

```
MacBook-Air-di-Valeria-10:gcc valerialukaj$ gcc -v -o hellogcc hellogcc.c
Apple clang version 13.1.6 (clang-1316.0.21.2.5)
Target: x86_64-apple-darwin21.2.0
Thread model: posix
InstalledDir: /Applications/Xcode.app/Contents/Developer/Toolchains/XcodeDefault.xctoolchain/usr/bin
 "/Applications/Xcode.app/Contents/Developer/Toolchains/XcodeDefault.xctoolchain/usr/bin/clang" -cc1 -triple x86_64-apple-macosx12.0.0 -Wundef-prefix=TARGET_OS_ -Wdeprecated-objc
-isa-usage -Werror=deprecated-objc-isa-usage -Werror=implicit-function-declaration -emit-obj -mrelax-all --mrelax-relocations -disable-free -disable-llvm-verifier -discard-value-
names -main-file-name hellogcc.c -mrelocation-model pic -pic-level 2 -mframe-pointer=all -fno-strict-return -fno-rounding-math -munwind-tables -target-sdk-version=12.3 -fvisibili
ty-inlines-hidden-static-local-var -target-cpu penryn -tune-cpu generic -debugger-tuning=lldb -target-linker-version 763 -v -resource-dir /Applications/Xcode.app/Contents/Develop
er/Toolchains/XcodeDefault.xctoolchain/usr/lib/clang/13.1.6 -isysroot /Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/MacOSX.sdk -I/usr/local/
include -internal-isystem /Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/MacOSX.sdk/usr/local/include -internal-isystem /Applications/Xcode.a
pp/Contents/Developer/Toolchains/XcodeDefault.xctoolchain/usr/lib/clang/13.1.6/include -internal-externc-isystem /Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platf
orm/Developer/SDKs/MacOSX.sdk/usr/include -internal-externc-isystem /Applications/Xcode.app/Contents/Developer/Toolchains/XcodeDefault.xctoolchain/usr/include -Wno-reorder-init-l
ist -Wno-implicit-int-float-conversion -Wno-c99-designator -Wno-final-dtor-non-final-class -Wno-extra-semi-stmt -Wno-misleading-indentation -Wno-quoted-include-in-framework-heade
r -Wno-implicit-fallthrough -Wno-enum-enum-conversion -Wno-enum-float-conversion -Wno-elaborated-enum-base -Wno-reserved-identifier -Wno-gnu-folding-constant -Wno-objc-load-metho
d -fdebug-compilation-dir=/Users/valerialukaj/Desktop/labs/gcc -ferror-limit 19 -stack-protector 1 -fstack-check -mdarwin-stkchk-strong-link -fblocks -fencode-extended-block-sign
ature -fregister-global-dtors-with-atexit -fgnuc-version=4.2.1 -fmax-type-align=16 -fcommon -fcolor-diagnostics -clang-vendor-feature=+messageToSelfInClassMethodIdReturnType -cla
ng-vendor-feature=+disableInferNewAvailabilityFromInit -clang-vendor-feature=+disableNonDependentMemberExprInCurrentInstantiation -fno-odr-hash-protocols -clang-vendor-feature=+e
nableAggressiveVLAFolding -clang-vendor-feature=+revert09abecef7bbf -clang-vendor-feature=+thisNoAlignAttr -clang-vendor-feature=+thisNoNullAttr -mllvm -disable-aligned-alloc-awa
reness=1 -D__GCC_HAVE_DWARF2_CFI_ASM=1 -o /var/folders/l3/xm_lwhm11jxdbq5sdl4httd80000gn/T/hellogcc-9fdcdc.o -x c hellogcc.c
clang -cc1 version 13.1.6 (clang-1316.0.21.2.5) default target x86_64-apple-darwin21.2.0
ignoring nonexistent directory "/Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/MacOSX.sdk/usr/local/include"
ignoring nonexistent directory "/Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/MacOSX.sdk/Library/Frameworks"
#include "..." search starts here:
#include <...> search starts here:
 /usr/local/include
 /Applications/Xcode.app/Contents/Developer/Toolchains/XcodeDefault.xctoolchain/usr/lib/clang/13.1.6/include
 /Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/MacOSX.sdk/usr/include
 /Applications/Xcode.app/Contents/Developer/Toolchains/XcodeDefault.xctoolchain/usr/include
 /Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/MacOSX.sdk/System/Library/Frameworks (framework directory)
End of search list.
 "/Applications/Xcode.app/Contents/Developer/Toolchains/XcodeDefault.xctoolchain/usr/bin/ld" -demangle -lto_library /Applications/Xcode.app/Contents/Developer/Toolchains/XcodeDef
ault.xctoolchain/usr/lib/libLTO.dylib -no_deduplicate -dynamic -arch x86_64 -platform_version macos 12.0.0 12.3 -syslibroot /Applications/Xcode.app/Contents/Developer/Platforms/M
acOSX.platform/Developer/SDKs/MacOSX.sdk -o hellogcc -L/usr/local/lib /var/folders/l3/xm_lwhm11jxdbq5sdl4httd80000gn/T/hellogcc-9fdcdc.o -lSystem /Applications/Xcode.app/Contents
/Developer/Toolchains/XcodeDefault.xctoolchain/usr/lib/clang/13.1.6/lib/darwin/libclang_rt.osx.a
MacBook-Air-di-Valeria-10:gcc valerialukaj$
```

Linux Compiler

When running GCC, use the `-v` option to see each step in detail.

```
$ gcc -v -o hellogcc hellogcc.c
```



Manually compile code

It can be useful to experience each step of compilation because, under some circumstances, you don't need GCC to go through all the steps.

First, delete the files generated by GCC in the current folder, except the source file.

```
$ rm a.out hellogcc.o
```

```
$ ls
```

```
hellogcc.c
```

Linux Compiler

Pre-processor

First, start the pre-processor, redirecting its output to `hellogcc.i`:

```
$ cpp hellogcc.c > hellogcc.i
```

```
$ ls
```

```
hellogcc.c  hellogcc.i
```

Take a look at the output file and notice how the pre-processor has included the headers and expanded the macros.

Linux Compiler

Compiler

Now you can compile the code into assembly. Use the `-S` option to set GCC just to produce assembly code.

```
$ gcc -S hellogcc.i
```

```
$ ls
```

```
hellogcc.c  hellogcc.i  hellogcc.s
```

Take a look at the assembly code to see what's been generated.

```
$ cat hellogcc.s
```

Linux Compiler

Assembly

Use the assembly code you've just generated to create an object file:

```
$ as -o hellogcc.o hellogcc.s
```

```
$ ls
```

```
hellogcc.c  hellogcc.i  hellogcc.o  hellogcc.s
```

Linux Compiler

Linking

To produce an executable file, you must link the object file to the libraries it depends on. This isn't quite as easy as the previous steps, but it's educational

```
$ ld -o hellogcc hellogcc.o
ld: warning: cannot find entry symbol _start; defaulting to
0000000000401000
ld: hellogcc.o: in function `main`:
hellogcc.c:(.text+0xa): undefined reference to `puts'
```

Linking

An error referencing an `undefined puts` occurs after the linker is done looking at the `libc.so` library. You must find suitable linker options to link the required libraries to resolve this. This is no small feat, and it's dependent on how your system is laid out.

When linking, you must link code to core runtime (CRT) objects, a set of subroutines that help binary executables launch. The linker also needs to know where to find important system libraries, including libc and libgcc, notably within special start and end instructions. These instructions can be delimited by the `--start-group` and `--end-group` options or using paths to `crtbegin.o` and `crtend.o`.

This example uses paths as they appear on a RHEL 8 install, so you may need to adapt the paths depending on your system.

Linux Compiler

```
$ ld -dynamic-linker \
/lib64/ld-linux-x86-64.so.2 \
-o hello \
/usr/lib64/crt1.o /usr/lib64/crti.o \
--start-group \
-L/usr/lib/gcc/x86 64-redhat-linux/8 \
-L/usr/lib64 -L/lib64 hello.o \
-lgcc \
--as-needed -lgcc s \
--no-as-needed -lc -lgcc \
--end-group
/usr/lib64/crtn.o
```